BRAC University
Department of Computer Science and Engineering
CSE470 – Software Engineering — Summer 2026

SOFTWARE REQUIREMENTS SPECIFICATION

for

ZENO

A Web-Based Parking Area Rental and Management System

Prepared By (Authors)

Student ID	Author Name
24241189	Abdullah Al Mazid Zomader
23201003	Masrur Sarar
22299250	Sraboni Din Awal
22299058	Arobi Al Amin Twinkle

Document Version: 1.0
Document Status: Initial Working Specification
Date: September 06, 2026

Contents

1 Introduction

1.1 Project Type

ZENO is a full-stack, web-based parking area rental and automated facility management platform developed using the modern MERN (MongoDB, Express.js, React, Node.js) technology stack. It operates as a centralized marketplace and operational utility for urban vehicle parking.

1.2 Purpose

The primary purpose of ZENO is to replace manual, error-prone paper logs and disjointed spreadsheets used by property managers with an automated digital ecosystem. It connects property owners who have underutilized parking slots with drivers who need secure, guaranteed parking. The system eliminates street congestion caused by circulating vehicles, automates payment settlement, and provides real-time visibility into slot occupancy. Furthermore, it enforces disciplined parking behavior through automated check-in monitoring and penalty calculations.

1.3 Target Users

ZENO is designed for three distinct user groups:

1. **Vehicle Drivers / Renters:** Commuters and visitors seeking short-term hourly parking or recurring monthly subscriptions near their destinations.
2. **Building Managers / Slot Owners:** Residential and commercial property owners who want to monetize vacant parking spaces, track occupancy, and monitor earnings.
3. **System Super-Administrators:** Platform operators responsible for verifying user credentials, overseeing payments, mediating disputes, and reviewing system-wide audit reports.

2 Functional Requirements

2.1 Map and Navigation

The Map and Navigation module provides an interactive visual representation of all registered parking premises using Leaflet mapping tools. Users can view nearby parking zones based on their current GPS coordinates or by panning across the city map. Each location pin displays the building name, exact distance, and number of available spots. Clicking a pin opens a detailed modal with driving directions and address guidance. The system updates marker states dynamically so drivers do not navigate toward fully occupied locations. This visual approach shortens search time and makes reaching the parking facility effortless.

2.2 Payment

The Payment feature delivers a frictionless and protected checkout experience for hourly bookings and overdue penalty fees. It integrates reliable digital payment gateways such as Stripe and SSLCommerz to handle debit cards, credit cards, and mobile financial services. When a driver confirms a reservation, the system computes the exact payable total including applicable taxes and processing fees. All sensitive transaction details are handled through encrypted channels without saving raw card numbers on the server. After payment confirmation, the transaction record is instantly stored in the database. Both the renter and the building owner receive immediate financial updates.

2.3 Subscription

The Subscription system caters to regular daily commuters seeking uninterrupted long-term parking privileges. Users can browse weekly or monthly recurring passes for specific residential or corporate complexes. The system automatically reserves a dedicated or guaranteed slot for the entire active subscription window. Automated billing cycles notify subscribers before their renewal date to avoid abrupt service loss. If a user cancels or changes plans, the platform updates slot availability immediately upon term expiration. This setup provides predictable revenue for property managers and peace of mind for daily drivers.

2.4 Reviews and Ratings

The Reviews and Ratings system establishes accountability and service quality across all registered parking facilities. Once a completed booking finishes, drivers are invited to submit a star rating alongside a written feedback comment. Renters can comment on lighting conditions, security personnel behavior, accessibility, and slot cleanliness. Building owners can view their aggregate satisfaction score and publicly reply to constructive feedback. Highly rated buildings receive greater prominence across general search listings. This feedback mechanism promotes transparency and helps new drivers choose trustworthy locations.

2.5 Invoice Generation

The Invoice Generation module automates legal financial documentation for every completed monetary transaction. Leveraging server-side PDFKit engines, the system compiles booking duration, slot identifier, building address, and total paid amounts into a clean PDF bill. Every invoice includes a unique serial number, timestamp, and breakdown of baseline fees versus penalties. Users can view and download invoices directly from their booking history dashboard at any time. Building managers can also generate consolidated billing statements for monthly accounting reviews. This built-in reporting guarantees compliance and eliminates manual bookkeeping errors.

2.6 Building Management

The Building Management module enables property administrators to register and maintain residential or commercial parking facilities. Administrators specify property names, physical street addresses, contact numbers, geo-coordinates, and operational guidelines. They can upload site entrance photos and define specific access rules, such as maximum vehicle height or operating hours. The module also allows managers to assign on-premise security supervisors to monitor daily gate operations. Any updates made to building metadata reflect across public renter search results immediately. This feature gives property administrators total digital authority over their physical facilities.

2.7 Slot Management

The Slot Management feature allows facility administrators to configure and control individual parking bays within each building floor. Administrators can define slot classifications, separating standard sedan spots from compact car, motorcycle, or electric vehicle charging slots. Each slot is given a distinct label, such as Floor-1 Slot A-04, to simplify on-site navigation for drivers. Administrators can mark individual slots as active, undergoing maintenance, or reserved for VIP residents. Real-time availability badges shift status as drivers complete reservations, check in, or vacate the spot. This level of granular control optimizes parking space utilization and prevents accidental overbooking.

2.8 Dynamic Pricing

The Dynamic Pricing engine allows building owners to optimize revenue based on shifting parking demand and specific time slots. Property managers can establish standard baseline hourly rates alongside customized weekend, peak-hour, or nighttime charges. When demand in a commercial district surges during business hours, the system can apply defined surge multipliers automatically. Conversely, off-peak discount rules can be enabled to attract drivers during historically quiet evenings. Renters always review transparent, pre-calculated pricing estimates before completing any checkout. This pricing flexibility maximizes property earnings while maintaining honest pricing transparency.

2.9 Search and Filter System

The Search and Filter system enables drivers to locate suitable parking spots matching their exact logistical criteria within seconds. Users can search by destination area, landmark name, or street address using responsive autocomplete suggestions. Filter options allow sorting by hourly price, vehicle compatibility, covered versus uncovered spaces, and user ratings. The results view switches smoothly between an interactive map perspective and a structured listing view. Inactive or fully booked facilities are flagged clearly so drivers do not waste time browsing unavailable choices. This focused discovery pipeline significantly reduces urban parking search frustration.

2.10 Bookings Feature

The Bookings module oversees the complete lifecycle of temporary vehicle parking reservations from start to finish. Drivers select arrival and departure times, pick an open parking bay, and confirm their booking with instant validation. The system temporarily locks selected slots during checkout to prevent duplicate concurrent reservations by different users. Once confirmed, the renter receives a booking reference code containing slot location and entry details. Renters can track active, upcoming, and past parking sessions through an organized central dashboard. This robust workflow ensures seamless coordination between arriving drivers and on-site attendants.

2.11 Overstay Detection and Penalty System

The Overstay Detection and Penalty System ensures fair parking turnover by discouraging drivers from exceeding their booked duration. The platform continuously compares active reservation end times against physical vehicle check-out records. When a vehicle remains parked past a designated grace period, the system flags the booking as an active overstay. Automatic incremental penalty charges are calculated according to the facility's published overstay rate policy. Renters receive automated alert warnings urging them to extend their reservation or vacate immediately. Outstanding fines must be cleared before the driver can initiate subsequent parking reservations.

2.12 Admin Control Panel

The Admin Control Panel serves as the central command dashboard for system superadministrators overseeing platform health. It aggregates high-level analytics, including total active reservations, platform-wide revenue volume, and total registered vehicles. Administrators possess authorization to verify newly registered building owners and audit suspicious user accounts. The control panel includes moderation tools to resolve disputed payments or investigate flagged user reviews. Real-time diagnostic logs and activity timestamps help administrators maintain operational security and platform integrity. This comprehensive console guarantees complete oversight over day-to-day platform activity.

2.13 Renter Profile and Vehicle Management

The Renter Profile and Vehicle Management module allows drivers to maintain accurate personal and vehicle credentials in one place. Users can save multiple vehicles under their profile by specifying license plate numbers, vehicle models, and categories. Having presaved vehicle profiles accelerates the checkout workflow by populating parking passes with verified vehicle data. Drivers can also update contact information, review saved payment methods, and monitor past reservation histories. Storing verified license plates helps security staff authenticate vehicles arriving at parking gates quickly. This self-service portal creates an organized and trustworthy environment for regular commuters.

2.14 Notification and Alert System

The Notification and Alert System keeps all platform participants informed of critical events through timely digital messages. Renters receive automated reminders thirty minutes prior to booking expiration to help them avoid inadvertent overstay penalties. Property managers receive instantaneous alerts whenever a new reservation, payment settlement, or cancellation occurs in their buildings. Notifications are dispatched via both in-app notification bells and automated email messages using Nodemailer services. Important administrative announcements regarding system maintenance or policy updates are pushed platform-wide. This reliable communication network keeps users updated without requiring constant manual page refreshes.

2.15 Reports and Export System

The Reports and Export System provides facility administrators with actionable financial and operational intelligence. Property managers can generate structured analytical summaries covering occupancy percentages, peak usage hours, and total revenue collections. Data can be filtered across custom date intervals, individual building floors, or specific parking slot types. The module allows managers to export clean tabular reports into standard CSV and Excel formats for offline audits. Visual bar and line charts make financial trends and revenue dips immediately apparent to stakeholders. These analytical tools empower property owners to make informed business decisions based on factual usage data.

3 Non-Functional Requirements

3.1 Performance

The platform must respond to user requests efficiently under ordinary and peak traffic loads. Server API endpoints should process standard search and booking requests within 500 milliseconds. Map marker rendering and filter updates must execute smoothly without perceptible frame stutter on the client browser. Database queries for slot occupancy verification must be optimized with appropriate indexes to prevent latency during checkout.

3.2 Security

Security and data protection are paramount across all operational layers of ZENO. All client-server communication must be securely encrypted via Transport Layer Security (HTTPS). User passwords must be securely hashed using salted bcrypt algorithms prior to database storage. Access to sensitive routes must be guarded by verified JSON Web Tokens (JWT) enforcing strict Role-Based Access Control (RBAC). Payment credentials must never be stored on internal application servers, adhering strictly to PCI-DSS standards via authorized gateways.

3.3 Usability

The user interface must follow modern user experience principles, prioritizing simplicity and visual clarity. Drivers must be able to complete a parking space reservation in fewer than four intuitive steps. Forms should provide clear, instantaneous inline validation messages to guide user input accurately. The layout must follow high-contrast readability standards, utilizing clean black-and-white typography and legible typography for effortless outdoor mobile viewing.

3.4 Reliability and Availability

ZENO must ensure high system reliability to support drivers needing round-the-clock parking access. The web application should maintain an operational uptime of at least 99.5% outside scheduled maintenance windows. Database transactions for slot reservations must follow ACID principles to avoid race conditions or phantom bookings. Automated daily database backups must be maintained to ensure rapid disaster recovery in case of hardware failures.

3.5 Maintainability and Scalability

The application codebase must be structured modularly, separating route handlers, business logic controllers, and database models. Both frontend and backend components should follow consistent coding conventions and clear architectural boundaries. The system must support horizontal scaling by deploying backend Express instances behind load balancers when traffic increases. MongoDB collections must be organized to allow smooth sharding as the platform expands into new geographical territories.

4 Class Diagram

The class diagram below outlines the core domain entities of the ZENO platform and their mutual associations. It details the structural relationships between Users, Vehicles, Buildings, Parking Slots, Bookings, Invoices, Payments, Subscriptions, and Penalty records.

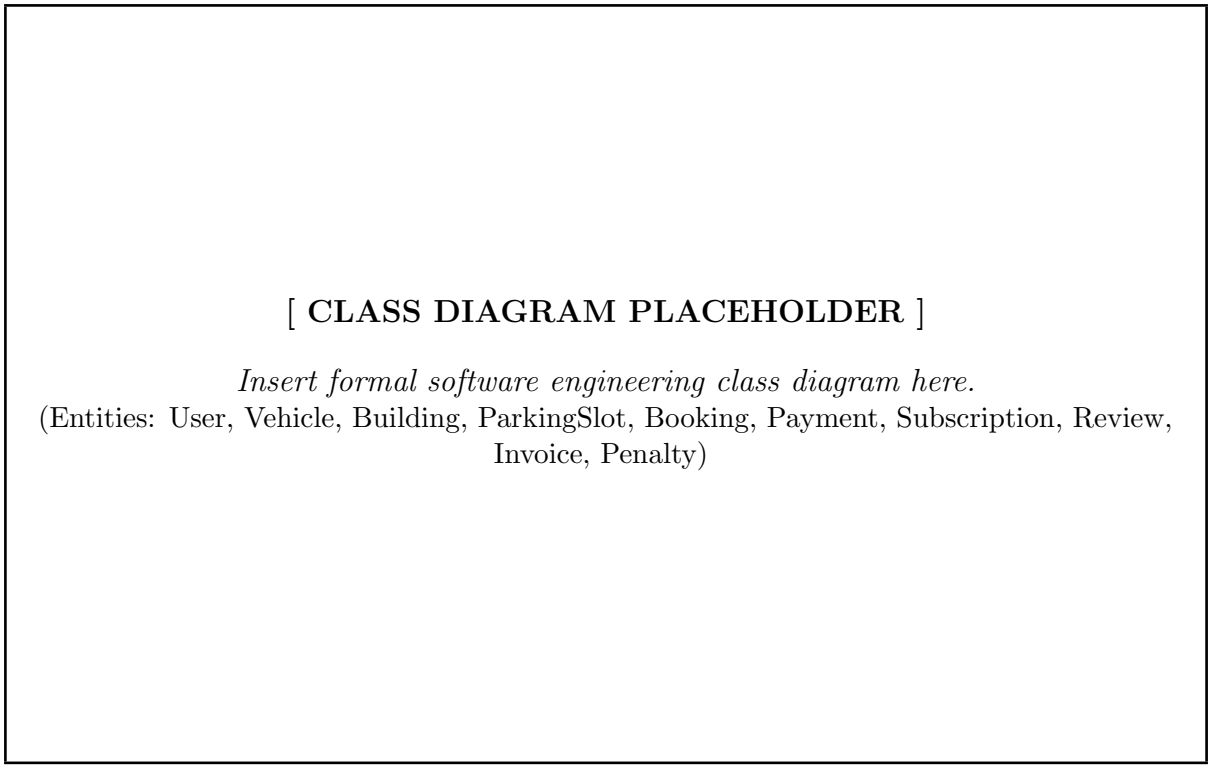

Figure 1: ZENO Domain Entity Class Diagram

5 Tools and Technologies

5.1 Frontend Technologies

- **React 19:** Component-based user interface library used to build a reactive and modular Single Page Application (SPA).
- **Vite:** High-speed next-generation frontend build tool providing rapid Hot Module Replacement (HMR) and optimized production bundles.
- **TailwindCSS:** Utility-first CSS styling framework used to produce responsive, consistent, and clean user interfaces.
- **React-Leaflet & Leaflet:** Open-source JavaScript mapping libraries utilized for interactive geospatial visualization, building location markers, and route previewing.
- **Axios:** Promise-based HTTP client for secure, asynchronous communication between the client browser and backend RESTful APIs.
- **Lucide React & Framer Motion:** Modern icon assets and fluid animation primitives delivering polished micro-interactions and clear visual feedback.
- **React-to-Print:** Specialized client-side utility for seamless printing and physical voucher formatting.

5.2 Backend Technologies

- **Node.js:** Scalable asynchronous event-driven JavaScript runtime environment executing high-throughput backend services.
- **Express.js 5:** Robust minimalist web application framework providing RESTful API routing, middleware chaining, and HTTP request handling.
- **MongoDB & Mongoose:** Document-oriented NoSQL database coupled with Mongoose Object Data Modeling (ODM) for strict schema definition and rapid query execution.
- **JSON Web Tokens (JWT):** Stateless authentication mechanism ensuring secure, signed session tokens for verified renters and administrators.
- **Bcryptjs:** Industrial-grade cryptographic hashing library protecting stored user credentials against rainbow table and brute-force intrusions.
- **PDFKit:** Dynamic server-side document generation engine that compiles transaction and booking data into downloadable PDF invoices.
- **Nodemailer:** Modular email delivery package responsible for dispatching automated booking confirmations, overstay alerts, and receipts.
- **Stripe / SSLCommerz Gateway Integrations:** Secure merchant payment processors facilitating encrypted credit card, debit card, and mobile banking transactions.

6 Compatibility with System Environment or OS

ZENO is designed with universal compatibility in mind, leveraging modern cross-platform web standards. On the client side, the platform operates independently of the client operating system. It runs flawlessly on all contemporary web browsers—including Google Chrome, Mozilla Firefox, Apple Safari, and Microsoft Edge—across desktop platforms (Windows 10/11, macOS, and Linux distributions) and mobile platforms (Android and iOS). This eliminates the friction of native app installations while ensuring responsive usability on screens of any dimensions.

On the server side, the backend runtime relies on Node.js, which guarantees smooth deployment across diverse enterprise server environments. It is fully compatible with standard Linux distributions (such as Ubuntu Server, Debian, and Red Hat Enterprise Linux), Windows Server environments, and containerized Docker environments. The database layer runs natively on local or cloud-managed MongoDB Atlas clusters, ensuring seamless infrastructure portability and consistent performance across production environments.

7 Implementation: Feature Screenshots

This section contains dedicated placeholders for application screenshots across all fifteen functional features of ZENO, accompanied by brief descriptions of their operational workflows.

7.1 Map and Navigation

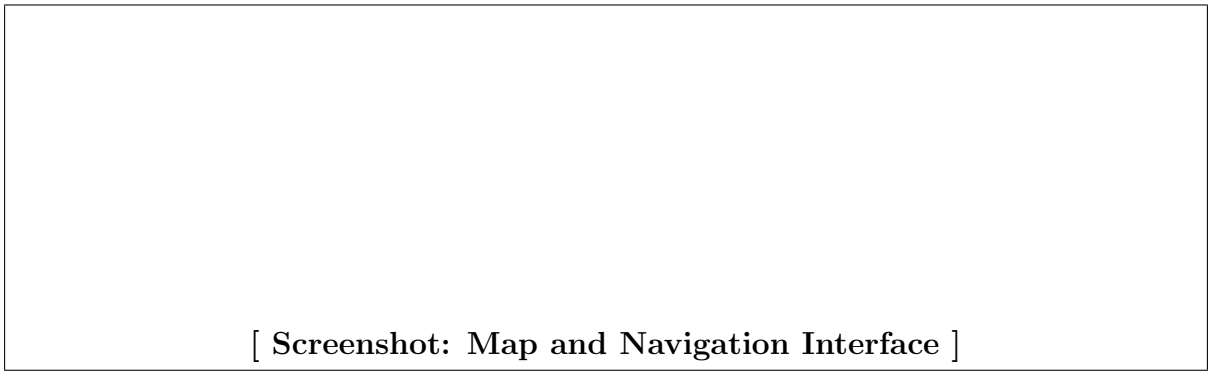

Figure 2: Interactive Map and Navigation Interface

Description: This interface displays an interactive city map with color-coded building markers indicating available parking locations. Users can click any marker to review building capacity and launch live turn-by-turn driving directions.

7.2 Payment

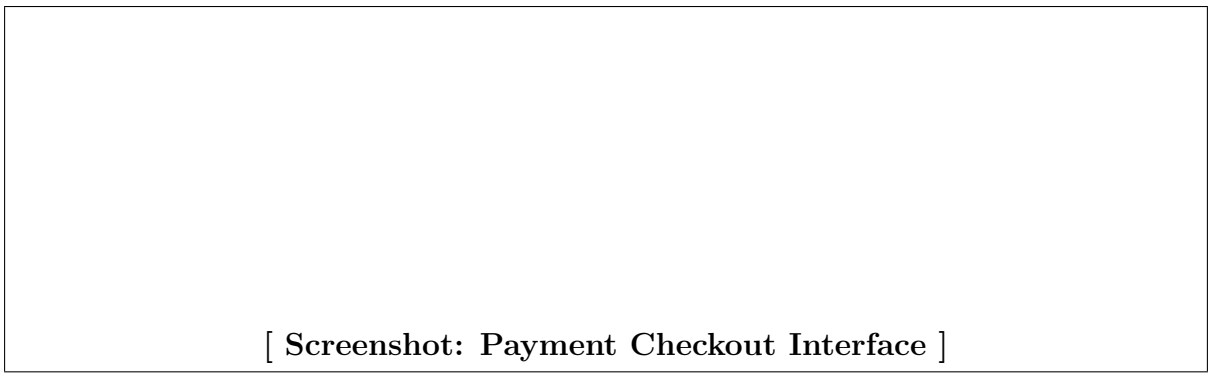

Figure 3: Secure Payment Checkout Screen

Description: The payment view presents a transparent fare breakdown alongside encrypted card and mobile banking checkout fields. It confirms transactions securely and instantly updates reservation statuses in real time.

7.3 Subscription

Figure 4: Recurring Subscription Management Screen

Description: This screen lists available weekly and monthly recurring parking passes for regular commuters. Drivers can activate, renew, or cancel long-term parking privileges with automated schedule tracking.

7.4 Reviews and Ratings

Figure 5: Community Reviews and Rating Screen

Description: This section allows verified drivers to submit star ratings and detailed comments regarding facility quality and security. Building managers can monitor overall customer sentiment and respond directly to user feedback.

7.5 Invoice Generation

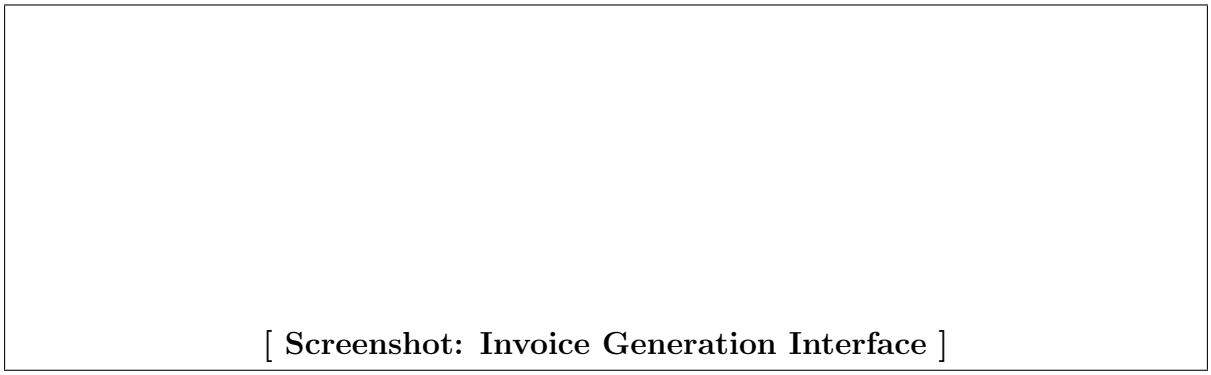

Figure 6: Automated PDF Invoice Document Screen

Description: This module displays the auto-generated digital invoice summarizing booking duration, fee items, and transaction IDs. Users can view the invoice online or download a formal PDF copy for personal records.

7.6 Building Management

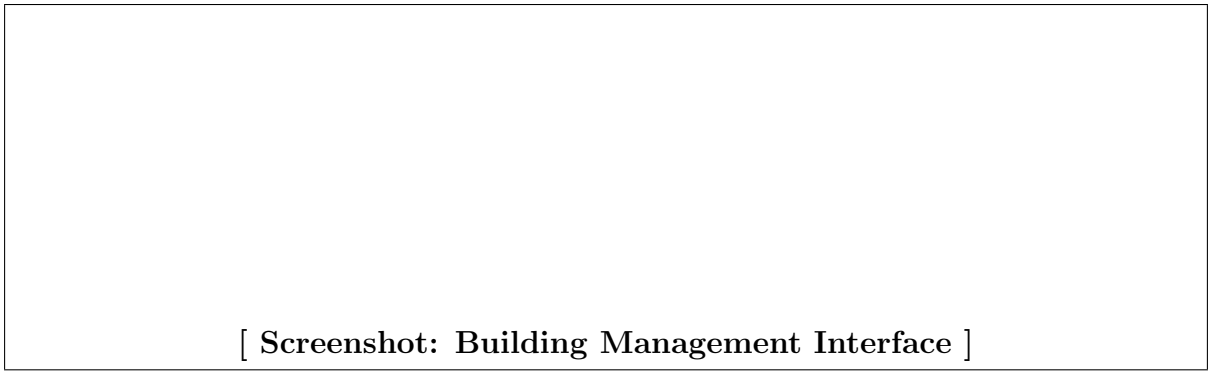

Figure 7: Building Registration and Administration Screen

Description: Property administrators use this form to register new facilities, input street addresses, and define operational guidelines. It centralizes all building profiles under an intuitive management interface.

7.7 Slot Management

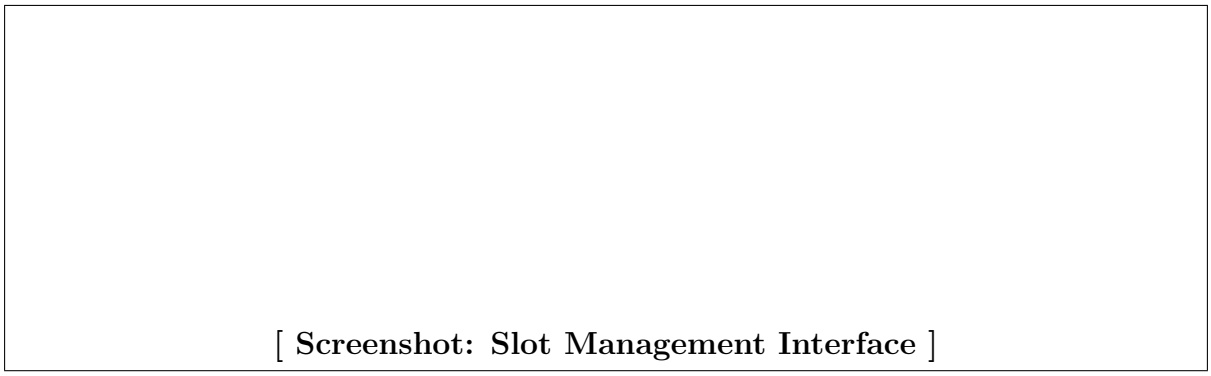

Figure 8: Parking Slot Grid and Status Controller

Description: This dashboard provides a visual grid of parking bays categorized by vehicle type and floor level. Administrators can quickly alter slot availability statuses or assign spots for scheduled maintenance.

7.8 Dynamic Pricing

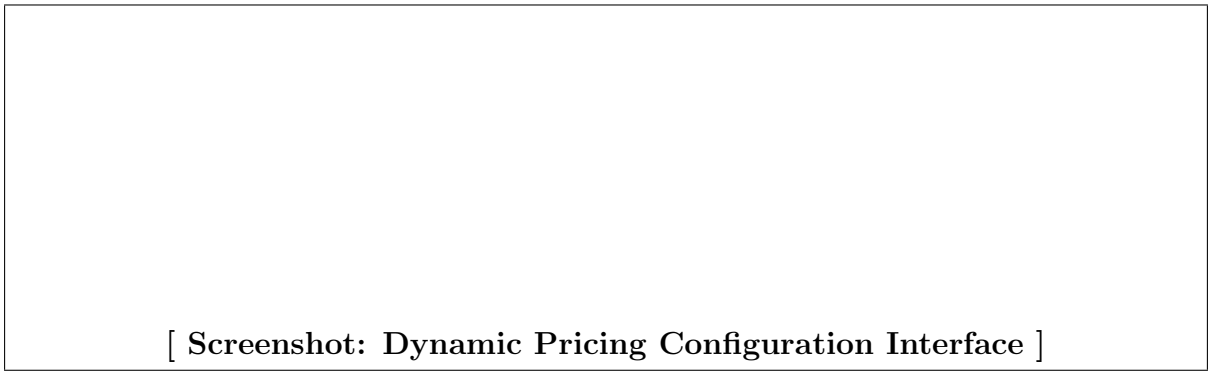

Figure 9: Dynamic Pricing Rules Configuration Screen

Description: This view enables managers to define base hourly parking rates, weekend adjustments, and rush-hour surge pricing. Changes apply automatically to upfront price estimates shown to searching renters.

7.9 Search and Filter System

[**Screenshot: Search and Filter System**]

Figure 10: Search and Multi-Criteria Filtering Interface

Description: The search bar allows drivers to find facilities by location, vehicle compatibility, budget constraints, and user ratings. Search results refresh dynamically without requiring full-page browser reloads.

7.10 Bookings Feature

[**Screenshot: Booking Reservation Interface**]

Figure 11: Active and Upcoming Bookings Manager

Description: Drivers select arrival and departure times, confirm slot selections, and receive instant booking verification vouchers. The dashboard organizes past, active, and upcoming parking sessions chronologically.

7.11 Overstay Detection and Penalty System

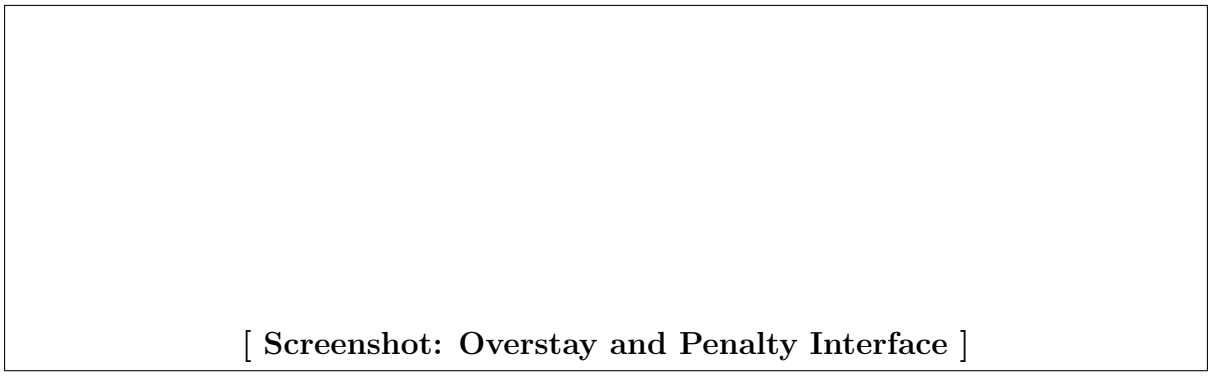

Figure 12: Overstay Tracking and Penalty Processing Screen

Description: This panel highlights overdue vehicles exceeding their reserved window and tallies automated penalty surcharges. It notifies both the driver and the facility attendant to facilitate rapid slot clearance.

7.12 Admin Control Panel

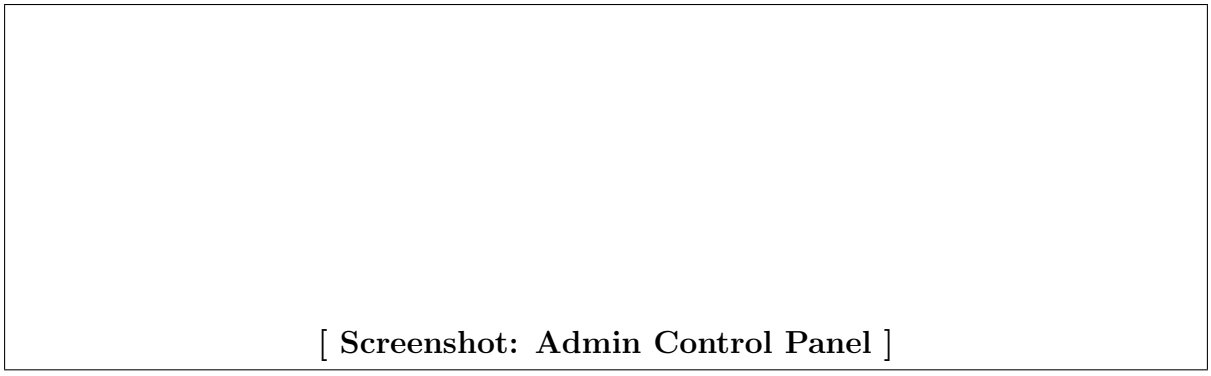

Figure 13: Central Super-Admin Command Dashboard

Description: The administrative dashboard summarizes platform-wide activity metrics, user verification queues, and dispute logs. Super-admins use this console to manage platform policies and audit financial flows.

7.13 Renter Profile and Vehicle Management

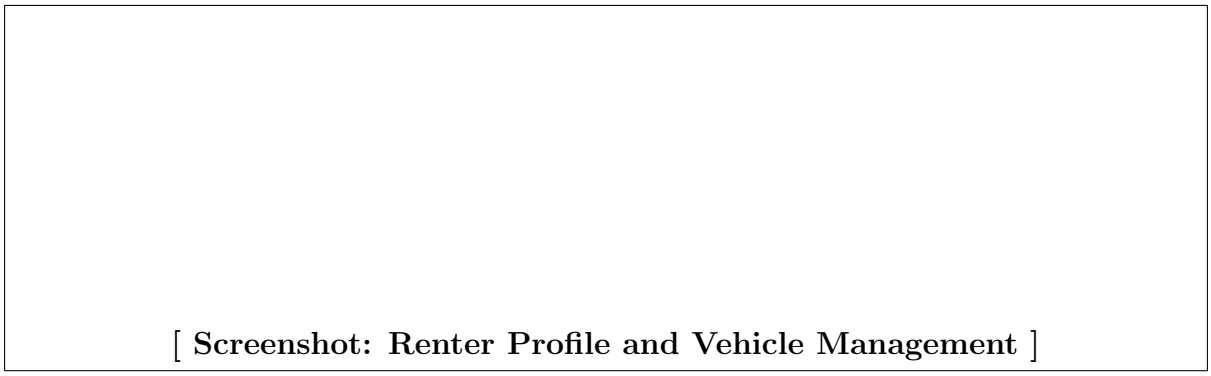

Figure 14: Driver Profile and Registered Vehicles Screen

Description: Users manage personal contact data and register multiple vehicle license plates for streamlined booking validation. Having pre-saved vehicle profiles ensures prompt entry authentication at facility gates.

7.14 Notification and Alert System

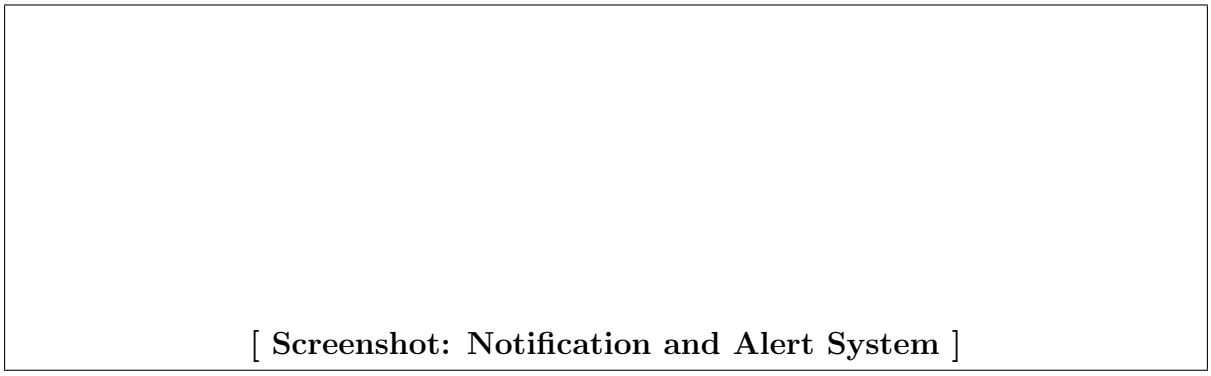

Figure 15: In-App Notifications and Alerts Drawer

Description: This drawer displays real-time updates regarding booking confirmations, payment completions, and upcoming session expirations. Critical notifications are also forwarded via automated email dispatch.

7.15 Reports and Export System

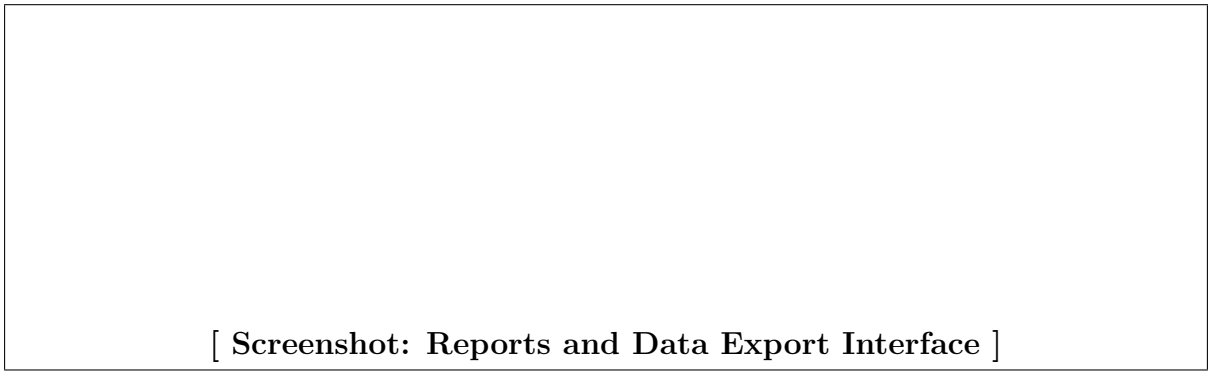

Figure 16: Analytical Reports and CSV/Excel Export Screen

Description: Property managers can review revenue trends and slot occupancy curves over custom time frames from this screen. Detailed transactional tables can be exported to CSV or Excel for accounting audits.

8 Challenges

Developing a real-time, multi-tenant parking marketplace introduces several technical and architectural hurdles. One significant challenge is preventing race conditions during concurrent bookings, where two drivers attempt to reserve the same parking slot simultaneously. This requires implementing transactional locking mechanisms at the database level to ensure atomic status updates.

Another demanding task is the automated overstay detection engine, which must run scheduled cron routines to compare checkout times without causing CPU bottlenecks. Additionally, integrating third-party map tiles and synchronizing location markers smoothly across varying network speeds demanded careful client-side state caching. Lastly, building a reliable server-side PDF generator capable of rendering consistent invoice layouts across international character sets required rigorous formatting precision.

9 Conclusion

ZENO provides an effective, full-stack digital solution for the widespread challenges of urban parking management. By connecting property owners with daily commuters through an intuitive web interface, it converts underutilized parking assets into productive revenue streams while removing driver parking stress. The platform consolidates map navigation, automated billing, transparent user reviews, and disciplined overstay monitoring into a cohesive workflow.

Its modular architecture guarantees that future enhancements—such as automated license plate recognition (ALPR) cameras, hardware IoT gate sensor integration, and predictive parking availability algorithms—can be integrated smoothly. Overall, ZENO establishes a modern foundation for smart city mobility and digitized space management.

10 References

[1] IEEE Computer Society, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.

[2] ISO/IEC/IEEE, *Systems and software engineering – Life cycle processes – Requirements engineering*, ISO/IEC/IEEE 29148:2018, 2018.

[3] React Documentation, *React – A JavaScript library for building user interfaces*, Meta Open Source, 2026. [Online]. Available: `https://react.dev/`

[4] Express.js Foundation, *Express – Fast, unopinionated, minimalist web framework for Node.js*, OpenJS Foundation, 2026. [Online]. Available: `https://expressjs.com/`

[5] MongoDB Inc., *The MongoDB Manual*, MongoDB Documentation, 2026. [Online]. Available: `https://www.mongodb.com/docs/`

[6] Leaflet Developers, *Leaflet – an open-source JavaScript library for mobile-friendly interactive maps*, 2026. [Online]. Available: `https://leafletjs.com/`

[7] Stripe Developers, *Stripe API Reference and Payment Integrations*, Stripe Inc., 2026. [Online]. Available: `https://stripe.com/docs`

[8] R. Pressman and B. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed., McGraw-Hill Education, 2020.